

CHAPTER

Building a REST API with Python

FastAPI · SQLAlchemy · Pydantic · Poetry · dotenv · pytest · Pathlib · PyYAML · REST Principles

Learning Objectives

By the end of this chapter, you will be able to:

- Understand the REST architectural style and HTTP method semantics.
- Set up a professional Python project using Poetry for dependency management.
- Manage environment-specific configuration securely with python-dotenv and .env files.
- Design and implement a relational database schema with SQLAlchemy ORM models.
- Build a production-ready REST API using FastAPI with path parameters, query parameters, and response models.
- Define and use Pydantic schemas for automatic request validation and response serialization.
- Apply best practices: PEP 8, type hints, mypy static type checking, and structured error handling.
- Write unit tests for API endpoints using pytest and FastAPI's TestClient.
- Read and parse files from the filesystem using Pathlib and regular expressions.
- Parse structured YAML configuration files using PyYAML.

SECTION 1 — THEORETICAL FOUNDATIONS

1.1 What is a REST API?

REST (**R**epresentational **S**tate **T**ransfer) is an architectural style for designing networked applications, introduced by Roy Fielding in his 2000 doctoral dissertation. A REST API is a web

service that exposes resources (data entities such as events, users, or products) over HTTP using a set of uniform, stateless operations. REST has become the dominant paradigm for web API design due to its simplicity, scalability, and compatibility with the stateless nature of HTTP.

In REST, every piece of data is a **resource** identified by a unique URL (the **endpoint**). Clients interact with resources using standard HTTP methods, and the server responds with a representation of the resource — almost always in JSON format in modern APIs. The server does **not store any client session state** between requests (**statelessness**) — every request must contain all information necessary for the server to process it.

1.1.1 HTTP Methods and REST Semantics

REST maps the four fundamental CRUD operations to HTTP methods:

HTTP Method	CRUD Operation	Typical Endpoint Example	Description
GET	Read	/events, /events/{id}	Retrieve a resource or list of resources. Must be safe (no side effects) and idempotent.
POST	Create	/events	Create a new resource. The server assigns the ID. Not idempotent (two identical POSTs create two records).
PUT	Update	/events/{id}	Replace an entire resource at the given ID with the request body. Idempotent.
PATCH	Update	/events/{id}	Partially update a resource — only the fields provided in the body are changed.
DELETE	Delete	/events/{id}	Remove the resource at the given ID. Idempotent — deleting a deleted resource returns 404, not an error.

1.1.2 HTTP Status Codes

REST APIs communicate the outcome of each request through standardised HTTP status codes:

Code	Meaning & Typical Use
200 OK	Request succeeded. Used for successful GET, PUT, PATCH.

201 Created	Resource successfully created. Used for successful POST.
204 No Content	Request succeeded but no body to return. Used for DELETE.
400 Bad Request	Client sent malformed or invalid data. Validation failure.
401 Unauthorized	Authentication credentials missing or invalid.
403 Forbidden	Credentials valid but insufficient permissions for this action.
404 Not Found	The requested resource does not exist.
422 Unprocessable Entity	Syntactically valid request but semantically invalid (Pydantic validation error in FastAPI).
500 Internal Server Error	An unexpected error occurred on the server side.

1.2 The Professional Python API Project Toolchain

A production-grade Python REST API project uses a carefully chosen set of tools that together enforce code quality, reproducibility, security, and testability. This chapter's project uses the following stack:

Tool / Library	Category	Purpose in This Project
FastAPI	API Framework	High-performance ASGI web framework for building REST APIs with automatic documentation.
SQLAlchemy	ORM	Maps Python classes to database tables; manages all database interactions.
Pydantic	Validation	Validates and serializes request/response data; integrated deeply into FastAPI.
Poetry	Dependency Manager	Manages packages, virtual environments, and project configuration in a unified tool.
python-dotenv	Configuration	Loads environment variables from .env files for secure, environment-specific config.
PyYAML	File Parsing	Parses YAML-format configuration and data files into Python dictionaries.
pytest	Testing	Runs unit and integration tests; integrates with FastAPI's TestClient.

mypy	Static Analysis	Checks type annotations statically to catch type errors before runtime.
Pathlib	Filesystem	Object-oriented filesystem path manipulation; used for file discovery.

1.3 Poetry — Modern Python Dependency Management

Poetry is a modern, all-in-one Python project management tool that addresses the shortcomings of the traditional `pip` + `requirements.txt` + `venv` workflow. Poetry manages three concerns in a single tool: **dependency declaration** (what your project needs), **dependency resolution** (finding compatible versions of all transitive dependencies), and **virtual environment management** (automatically creating and activating an isolated environment).

Poetry uses two files. The `pyproject.toml` file is the project's manifest — it declares direct dependencies with version constraints (e.g., `fastapi = "^0.110.0"`). The `poetry.lock` file is the resolved lockfile — it records the **exact version** of every package (direct and transitive) that was installed. Committing the lockfile to version control guarantees that every developer and CI/CD pipeline installs **identical** dependency versions — the foundation of reproducible builds.

Command	Purpose
<code>poetry new project-name</code>	Scaffold a new Poetry project with standard directory structure.
<code>poetry install</code>	Install all dependencies declared in <code>pyproject.toml</code> (or exact versions from <code>poetry.lock</code> if it exists).
<code>poetry add fastapi</code>	Add a new dependency, update <code>pyproject.toml</code> and <code>poetry.lock</code> .
<code>poetry add --dev pytest</code>	Add a development-only dependency (not installed in production).
<code>poetry run python file.py</code>	Execute a Python script inside Poetry's managed virtual environment.
<code>poetry shell</code>	Activate Poetry's virtual environment in the current shell session.
<code>poetry update</code>	Update all dependencies to the latest compatible versions and regenerate the lockfile.
<code>poetry show</code>	List all installed packages and their resolved versions.

1.4 FastAPI — High-Performance Python API Framework

FastAPI is a modern, high-performance Python web framework for building REST APIs. It is built on top of **Starlette** (a lightweight ASGI toolkit) for performance and **Pydantic** for data validation. FastAPI is one of the fastest Python web frameworks available — on par with Node.js and Go — because it is built on Python's **asyncio** infrastructure and does not rely on blocking I/O.

FastAPI's distinguishing features compared to older frameworks like Flask and Django REST Framework are:

- **Automatic API Documentation** — FastAPI generates interactive Swagger UI (at `/docs`) and ReDoc (at `/redoc`) documentation automatically from your code — no manual documentation required.
- **Native Type Hint Integration** — FastAPI reads Python type annotations on function parameters and uses them to perform automatic request parsing, validation, and serialization.
- **Pydantic Integration** — Request bodies and response models are defined as Pydantic models, providing automatic validation, error messages, and JSON serialization.
- **Async-First Design** — FastAPI fully supports Python's **async def** endpoint functions for non-blocking I/O operations (database queries, external API calls).
- **Path and Query Parameters** — Route parameters (`/events/{event_id}`) are automatically parsed and type-converted based on function signature type hints.

1.4.1 FastAPI vs Flask vs Django REST Framework

Dimension	FastAPI	Flask	Django REST Framework
Performance	Excellent (async-native, ASGI)	Good (WSGI, sync by default)	Good (WSGI, ORM overhead)
Auto Docs	Built-in (Swagger + ReDoc)	Manual (Flask-RESTX or similar)	Built-in DRF browsable API
Validation	Pydantic (native, automatic)	Manual or Marshmallow	DRF Serializers
Type Hints	First-class citizen	Optional, not integrated	Optional, not integrated
Learning Curve	Moderate	Low	High (Django ORM, DRF concepts)
Best For	Microservices, AI/ML APIs, high-perf	Simple APIs, prototypes	Full-featured web apps with admin

1.5 Pydantic — Data Validation and Serialization

Pydantic is Python's most popular data validation library. At its core, Pydantic allows you to define data shapes as Python classes (inheriting from `BaseModel`) with type-annotated fields. When you create an instance of a Pydantic model, it **automatically validates** the provided data against the declared types and constraints, raising a `ValidationError` with detailed, human-readable error messages if the data is invalid.

In the FastAPI context, Pydantic models serve two distinct roles. As **request schemas**, they validate and parse incoming JSON request bodies. As **response schemas**, they control exactly what data is serialized into the JSON response — preventing accidental exposure of sensitive fields (like password hashes) that exist in the SQLAlchemy model but should not be returned to clients. The `response_model` parameter on a FastAPI endpoint decorator accepts a Pydantic model and enforces the response structure.

1.5.1 The Separation of DB Models and Pydantic Schemas

A critical architectural pattern in FastAPI applications is maintaining a clear separation between **SQLAlchemy models** (which define the database schema and are used internally) and **Pydantic schemas** (which define the API contract — what the client sees). The naming convention `DBEvent` (database model) vs `EventSchema` (API schema) makes this distinction explicit. This separation enables:

- Evolving the database schema independently of the API contract.
- Selectively exposing only safe, relevant fields to API clients.
- Applying different validation rules for reading vs. writing data.
- Maintaining clean, testable domain models.

1.6 Environment Configuration with python-dotenv

Hardcoding configuration values — database connection strings, API keys, passwords, service URLs — directly in source code is a critical security anti-pattern. When code is committed to version control (Git), hardcoded secrets become part of the commit history and may be exposed if the repository is ever made public or if unauthorized access occurs.

The industry-standard solution is the **.env file pattern**, formalised by the Twelve-Factor App methodology. Configuration values are stored in a `.env` file (which is added to `.gitignore` and never committed to version control). The `python-dotenv` library reads this file at application startup and populates Python's `os.environ` dictionary, making the values available via `os.getenv()`. Different environments (development, staging, production) maintain separate `.env` files with environment-appropriate values.

Security Rule: Never Commit .env Files

Always add `.env` to `.gitignore` before making the first commit. Instead, provide a `.env.example` file with placeholder values that **is** committed to the repository — this documents what configuration is needed without exposing actual secrets. In production, use your platform's secret management system (AWS Secrets Manager, Kubernetes Secrets, Heroku Config Vars) rather than `.env` files.

1.7 Database Design — Events and Categories

The project manages a system for tracking events (such as ticketed sports games, concerts, or conferences) organized by category. The database schema consists of two related tables: **category** (the parent) and **event** (the child). This models a classic **one-to-many relationship**: one category (e.g., 'Sports') can contain many events, but each event belongs to exactly one category.

1.7.1 Schema Design Principles Applied

The schema in this project demonstrates several important relational design principles:

- **INTEGER PRIMARY KEY** — Every table has a surrogate primary key (an auto-incrementing integer) that uniquely identifies each row. This is more stable than using business data (like a product code) as a primary key, since business data can change.
- **UNIQUE constraint on product_code** — The `product_code` column is declared UNIQUE, preventing duplicate event entries and making it a reliable natural identifier for API lookups.
- **NOT NULL constraints** — Critical fields (date, price, category_id) are declared NOT NULL, enforcing data integrity at the database level — not just at the application layer.
- **Foreign Key Constraint** — The `category_id` column in the `event` table REFERENCES the `category` table, ensuring referential integrity: you cannot insert an event with a category_id that doesn't exist in the category table.
- **NUMERIC type for price** — Financial values should always use NUMERIC (or DECIMAL) rather than FLOAT. Floating-point arithmetic is approximate — $0.1 + 0.2 \neq 0.3$ in binary floating point. NUMERIC stores decimal values exactly.

1.8 Professional Best Practices

The project enforces several professional coding standards that distinguish production-ready code from quick prototypes:

1.8.1 PEP 8 — The Python Style Guide

PEP 8 is Python's official style guide. It covers naming conventions (`snake_case` for functions and variables, `PascalCase` for classes), maximum line length (79 characters for code), spacing around operators, import ordering, and documentation strings. Consistent adherence to PEP 8 makes codebases readable by any Python developer and enables automated tools like `flake8` and `black` to check and enforce the style automatically.

1.8.2 Type Hints and mypy

Python 3.5+ supports optional type annotations (**type hints**) via PEP 484. While Python's runtime does not enforce types, type hints enable **static analysis tools** like `mypy` to catch type mismatches before the code is ever executed — similar to how a compiled language would catch type errors at compile time. In FastAPI, type hints are not optional — they are the mechanism by which FastAPI understands your endpoint's parameters and response types.

1.8.3 Pathlib — Object-Oriented Filesystem Access

Python's `pathlib` module (introduced in Python 3.4) provides an object-oriented approach to filesystem path manipulation, replacing the older `os.path` string-based API. A `Path` object represents a filesystem path and provides intuitive methods for common operations: `.exists()`, `.glob()` (pattern-based file discovery), `.read_text()`, `.write_text()`, `.mkdir()`, and the `/` operator for joining paths (`Path('data') / 'events.yaml'`).

1.8.4 Regular Expressions

Regular expressions (regex) are patterns that describe sets of strings. Python's `re` module provides full regex support. In the context of this project, regex is used to extract structured information from filenames or file content — for example, identifying files matching the pattern `EVENT-\d+` (EVENT- followed by one or more digits) in a directory of YAML files. Key functions: `re.search()` (find first match anywhere in string), `re.match()` (match from start of string), `re.findall()` (find all non-overlapping matches).

1.8.5 PyYAML — Parsing YAML Configuration Files

YAML (YAML Ain't Markup Language) is a human-readable data serialization format widely used for configuration files (`docker-compose.yml`, `kubernetes` manifests, GitHub Actions workflows). PyYAML's `yaml.safe_load()` function parses a YAML string or file into native Python objects (dictionaries, lists, strings, numbers, booleans). `safe_load` is the secure variant — use it instead of `yaml.load()` to prevent arbitrary Python object instantiation from untrusted YAML content.

SECTION 2 — PRACTICAL & CODING REFERENCE

This section walks through every component of the REST API project with complete, annotated code cells. Each cell includes the full code, the conceptual explanation, the rationale for the

approach, and a detailed output walkthrough. The cells are organized to reflect the natural build order of the project.

Part A — Project Setup and Configuration

Cell 1 — Poetry Project Setup and Dependency Management

```
# Step 1: Create a new Poetry project
poetry new events-api

# The generated project structure:
# events-api/
# └─ pyproject.toml      <- project manifest
# └─ poetry.lock         <- resolved lockfile (generated after install)
# └─ README.md
#   └─ events_api/
#     └─ __init__.py

# Step 2: Add runtime dependencies
poetry add fastapi
poetry add sqlalchemy
poetry add python-dotenv
poetry add pyyaml
poetry add uvicorn      # ASGI server to run FastAPI

# Step 3: Add development-only dependencies
poetry add --dev pytest
poetry add --dev httpx   # required by FastAPI TestClient
poetry add --dev mypy

# Step 4: Install all dependencies
poetry install

# Step 5: Run the API server
poetry run python runserver.py
# OR: poetry run uvicorn events_api.main:app --reload
```

Concept

Poetry's `new` command scaffolds a complete project structure with a `pyproject.toml` manifest. The `add` command adds a dependency to `[tool.poetry.dependencies]` in `pyproject.toml` and resolves its compatible version automatically. `add --dev` adds to `[tool.poetry.dev-dependencies]` — these packages are installed locally for development but excluded from production deployments. After running `poetry install`, a `poetry.lock` file is generated containing the exact resolved versions of every package, including transitive dependencies. Uvicorn is an ASGI server — it handles the actual HTTP connections and forwards requests to FastAPI.

Why This Approach?

Poetry eliminates the 'it works on my machine' problem. By committing `poetry.lock` to version control, every team member and CI/CD pipeline installs **byte-for-byte identical** environments. The `--dev` flag separates testing and linting tools from production dependencies — production Docker images can run `poetry install --no-dev` to produce a leaner image. The `--reload` flag on uvicorn watches the source directory for changes and restarts automatically during development.

Expected Output & Explanation

Created package events-api at events-api/ (project scaffolded)
 Using version ^0.110.0 for fastapi (Poetry selects latest compatible version)
 Package operations: Installing 12 packages (FastAPI + all transitive dependencies)
 Updating poetry.lock (exact resolved versions recorded)
 INFO: Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit) (server started)

Cell 2 — Environment Configuration with python-dotenv

```
# --- .env file (NEVER commit to git) ---
# DATABASE_URL="sqlite:///events.db"
# API_SECRET_KEY="your-super-secret-key-here"
# ENVIRONMENT="development"
# DEBUG="true"

# --- .env.example file (DO commit to git) ---
# DATABASE_URL=""
# API_SECRET_KEY=""
# ENVIRONMENT="development"

# --- config.py ---
```

```
import os
from dotenv import load_dotenv
from pathlib import Path

# Load .env file from project root
BASE_DIR = Path(__file__).resolve().parent.parent
load_dotenv(BASE_DIR / ".env")

# Read configuration values
DATABASE_URL = os.getenv("DATABASE_URL", "sqlite:///events.db")
DEBUG = os.getenv("DEBUG", "false").lower() == "true"
ENVIRONMENT = os.getenv("ENVIRONMENT", "development")

# Validate critical configuration at startup
if not DATABASE_URL:
    raise ValueError("DATABASE_URL must be set in .env file")

print(f"Environment : {ENVIRONMENT}")
print(f"Database : {DATABASE_URL}")
print(f"Debug mode : {DEBUG}")
```

Concept

The `load_dotenv()` call reads the `.env` file and injects each key-value pair into `os.environ` — making them accessible via `os.getenv()` throughout the application. The second argument to `os.getenv()` is a **default value** — returned when the variable is not set. `Path(__file__).resolve().parent.parent` navigates from the config file's location up to the project root directory, making the `.env` path absolute regardless of from where the application is launched. The startup validation (`raise ValueError if not DATABASE_URL`) follows the **fail-fast principle** — if critical config is missing, crash immediately with a clear message rather than failing mysteriously at runtime.

Why This Approach?

This pattern fully separates code (committed to git) from configuration (never committed). The `.env.example` file serves as living documentation of what configuration variables the application expects — developers cloning the repo know exactly what to set up. The default value pattern (`os.getenv('KEY', 'default')`) ensures the application can run in a development environment with sensible defaults even without a `.env` file. `Path(__file__)` makes the config loading path-independent — the application can be launched from any directory.

Expected Output & Explanation

Environment : development

Database : sqlite:///events.db

Debug mode : false (or true if DEBUG=true in .env)

If DATABASE_URL is missing: ValueError: DATABASE_URL must be set in .env file (fail-fast)

Part B — Database Design and SQLAlchemy Models

Cell 3 — SQL Schema — Category and Event Tables

```
-- =====
-- DDL: Data Definition Language for the events-api database
-- =====

-- Parent table: Category
CREATE TABLE category (
    id    INTEGER PRIMARY KEY AUTOINCREMENT,
    name  VARCHAR(64) NOT NULL UNIQUE
);

-- Child table: Event
CREATE TABLE event (
    id            INTEGER PRIMARY KEY AUTOINCREMENT,
    product_code  VARCHAR(32) NOT NULL UNIQUE,
    date          DATE        NOT NULL,
    price         NUMERIC(12, 2) NOT NULL,
    category_id   INTEGER     NOT NULL,

    -- Foreign key: each event belongs to one category
    CONSTRAINT fk_event_category
        FOREIGN KEY (category_id)
            REFERENCES category(id)
            ON DELETE RESTRICT    -- prevent deleting a category with events
);

-- Indexes for common query patterns
```

```
CREATE INDEX idx_event_category ON event(category_id);
CREATE INDEX idx_event_date ON event(date);
```

Concept

This schema implements a **one-to-many relationship**: one row in `category` may be referenced by many rows in `event` via the `category_id` foreign key. `NOT NULL UNIQUE` on `category.name` prevents duplicate category names. `NOT NULL UNIQUE` on `event.product_code` ensures each event has a unique business identifier. `NUMERIC(12,2)` stores price with exactly 2 decimal places — critical for financial accuracy. `ON DELETE RESTRICT` prevents deleting a category if any events reference it, protecting data integrity. Indexes on `category_id` and `date` accelerate the most common query patterns (filter events by category, filter by date range).

Why This Approach?

Schema design decisions have permanent consequences — migrating a production database is costly and risky. Choosing `NUMERIC` over `FLOAT` for price prevents cumulative rounding errors in financial calculations. The `ON DELETE RESTRICT` constraint enforces a business rule at the database level: a category cannot be deleted while it still has events. This is more reliable than enforcing the rule only in application code, where a direct database connection bypassing the API could violate it. Indexes are a performance investment: they add overhead to `INSERT/UPDATE` but dramatically accelerate `SELECT` queries on large tables.

Expected Output & Explanation

Schema created. Tables: `category (id, name)`, `event (id, product_code, date, price, category_id)`.

Foreign key constraint ensures: every `event.category_id` must reference an existing `category.id`.

Indexes created on `event.category_id` and `event.date` for query optimization.

Cell 4 — SQLAlchemy ORM Models — DBCategory and DBEvent

```
# models.py
import decimal
from datetime import date
from typing import List, Optional
from sqlalchemy import ForeignKey, String, Numeric, Date
from sqlalchemy.orm import DeclarativeBase, Mapped, mapped_column,
relationship
```

```
class Base(DeclarativeBase):
    """Base class for all ORM models in this project."""
    pass

class DBCategory(Base):
    """ORM model for the 'category' table."""
    __tablename__ = "category"

    id: Mapped[int] = mapped_column(primary_key=True)
    name: Mapped[str] = mapped_column(String(64), unique=True,
    nullable=False)

    # One-to-many: one category -> many events
    events: Mapped[List["DBEvent"]] = relationship(
        back_populates="category",
        cascade="all, delete-orphan",
        lazy="select"
    )

    def __repr__(self) -> str:
        return f"DBCategory(id={self.id}, name={self.name!r})"

class DBEvent(Base):
    """ORM model for the 'event' table."""
    __tablename__ = "event"

    id: Mapped[int] = mapped_column(primary_key=True)
    product_code: Mapped[str] = mapped_column(String(32),
    unique=True, nullable=False)
    date: Mapped[date] = mapped_column(Date,
    nullable=False)
    price: Mapped[decimal.Decimal] = mapped_column(Numeric(12, 2),
    nullable=False)
    category_id: Mapped[int] = mapped_column(
        ForeignKey("category.id", ondelete="RESTRICT"), nullable=False
    )
```

```

# Many-to-one: each event belongs to one category
category: Mapped["DBCategory"] = relationship(
    back_populates="events"
)

def __repr__(self) -> str:
    return (
        f"DBEvent(code={self.product_code!r}, "
        f"price={self.price}, date={self.date})"
    )

```

Concept

The **DB** prefix (**DBCategory** , **DBEvent**) is a naming convention that visually distinguishes SQLAlchemy database models from Pydantic API schemas (**CategorySchema** , **EventSchema**). **Mapped[List["DBEvent"]]** is SQLAlchemy 2.0's type-annotated way to declare a one-to-many relationship — the string literal **"DBEvent"** is a forward reference allowing the class to be referenced before it is defined. **lazy="select"** means related objects are loaded in a separate SELECT query when first accessed (**lazy loading**), rather than being joined in the initial query. **Mapped[decimal.Decimal]** maps to SQLAlchemy's **Numeric** column type, which uses Python's **decimal.Decimal** for exact arithmetic.

Why This Approach?

Separating models (**models.py**) from schemas (**schemas.py**) from routes (**routers/events.py**) follows the **separation of concerns** principle. It makes the codebase modular — you can change the database schema without touching the API layer and vice versa. The **Numeric(12,2)** mapping ensures Python's **decimal.Decimal** is used for price arithmetic — never **float** for money. The **cascade='all, delete-orphan'** on the Category side means deleting a **DBCategory** object also deletes all its associated **DBEvent** objects via SQLAlchemy, before the database-level **RESTRICT** constraint even fires.

Expected Output & Explanation

No visual output — models define the schema structure. With **echo=True** on **create_engine**:

```

CREATE TABLE category (id INTEGER NOT NULL, name VARCHAR(64) NOT NULL
UNIQUE, PRIMARY KEY (id))
CREATE TABLE event (id INTEGER NOT NULL, product_code VARCHAR(32) NOT NULL,
date DATE NOT NULL, price NUMERIC(12,2) NOT NULL, category_id INTEGER NOT
NULL, PRIMARY KEY (id), FOREIGN KEY(category_id) REFERENCES category(id))

```

Cell 5 — Database Connection Engine and Session Factory

```
# database.py

from sqlalchemy import create_engine
from sqlalchemy.orm import sessionmaker, Session
from config import DATABASE_URL
from models import Base


# Create the engine - connection factory to the database
engine = create_engine(
    DATABASE_URL,
    echo=False,          # Set True in development to log SQL
    connect_args={
        "check_same_thread": False # Required for SQLite with FastAPI
    }
)

# Session factory - produces new Session instances
SessionLocal = sessionmaker(
    autocommit=False,      # Transactions are explicit
    autoflush=False,      # Don't flush before every query
    bind=engine
)

# Create all tables (call once at startup)
def create_tables() -> None:
    Base.metadata.create_all(bind=engine)


# FastAPI Dependency: provides a database session per request
def get_db():
    """
    Yield a database session for the duration of a single request.
    Closes the session automatically when the request completes.
    """
    db: Session = SessionLocal()
    try:
```



```

yield db

finally:

    db.close()

```

Concept

The `engine` is the core connection factory — created once at application startup and shared across all requests. The `check_same_thread=False` argument is a SQLite-specific requirement: by default, SQLite only allows connections to be used from the thread that created them. FastAPI processes requests asynchronously across multiple threads, so this restriction must be disabled. `SessionLocal` is a session **factory** — calling `SessionLocal()` creates a new `Session` instance. The `get_db()` function is a **FastAPI dependency** — it uses Python's `yield` to provide a session for the duration of a request and guarantees `db.close()` runs in the `finally` block even if an exception occurs, preventing connection leaks.

Why This Approach?

The `get_db()` dependency pattern is the standard FastAPI database session management approach. FastAPI's `Depends()` system calls `get_db()` before the endpoint handler and tears it down after — each HTTP request gets its own isolated database session. `autocommit=False` means database changes are not persisted until `session.commit()` is explicitly called — this gives the endpoint handler full control over transaction boundaries. If the handler raises an exception, the session is closed without committing, so the database remains unchanged.

Expected Output & Explanation

Database engine created: ENGINE at sqlite:///events.db
 create_tables() call issues: CREATE TABLE IF NOT EXISTS category ...; CREATE TABLE IF NOT EXISTS event ...;
 get_db() provides a fresh Session per request and closes it in the finally block — guaranteed cleanup regardless of success or error.

Part C — FastAPI Application and API Endpoints

Cell 6 — FastAPI Application, Root Endpoint, and Pydantic Schemas

```

# schemas.py — Pydantic models for API validation and serialization

from datetime import date

from decimal import Decimal

from pydantic import BaseModel, ConfigDict

```

```
class CategorySchema(BaseModel):
    id: int
    name: str

    model_config = ConfigDict(from_attributes=True) # Pydantic v2

class EventSchema(BaseModel):
    id: int
    product_code: str
    date: date
    price: Decimal
    category_id: int

    model_config = ConfigDict(from_attributes=True)

class EventDetailSchema(EventSchema):
    """Extended schema that includes the nested category details."""
    category: CategorySchema

# main.py - FastAPI application entry point
from fastapi import FastAPI
from database import create_tables

app = FastAPI(
    title="Events API",
    description="REST API for managing events and categories.",
    version="1.0.0"
)

# Create tables on startup
@app.on_event("startup")
def on_startup():
    create_tables()
```

```
# Root endpoint — health check
@app.get("/", tags=["Health"])
def home() -> dict:
    return {
        "message": "Events API is running",
        "version": "1.0.0",
        "docs":    "/docs"
    }
```

Concept

The `model_config = ConfigDict(from_attributes=True)` setting (Pydantic v2) allows Pydantic to read data from SQLAlchemy ORM objects using attribute access (`event.product_code`) rather than requiring a plain dictionary — this is the bridge between the ORM layer and the API serialization layer. `EventDetailSchema` inherits from `EventSchema` and adds a nested `CategorySchema` — this demonstrates Pydantic's composition model. The `@app.on_event('startup')` decorator registers a function to run once when the server starts. The root endpoint serves as a **health check** — a simple endpoint that confirms the API is running, commonly pinged by load balancers and monitoring systems. The FastAPI constructor's `title` and `description` appear in the auto-generated Swagger documentation.

Why This Approach?

The Pydantic-SQLAlchemy bridge via `from_attributes=True` is the critical configuration that makes `response_model` work. Without it, returning a SQLAlchemy `DBEvent` object where FastAPI expects an `EventSchema` would raise a `ValidationError`. `EventDetailSchema` inheriting from `EventSchema` demonstrates **schema inheritance** — the detail view adds the nested category without duplicating field definitions. The `/docs` URL in the health check response guides API consumers to the interactive documentation — a developer-friendly touch.

Expected Output & Explanation

GET / response: {"message": "Events API is running", "version": "1.0.0", "docs": "/docs"}

GET /docs: Interactive Swagger UI with all endpoints listed, schemas shown, try-it-out enabled.

Startup log: INFO: Application startup complete. / INFO: CREATE TABLE category ... (with echo=True)

Cell 7 — REST Endpoints — Get All Events and Get Single Event

```
# routers/events.py
```

```
from typing import List
from fastapi import APIRouter, Depends, HTTPException, status
from sqlalchemy.orm import Session
from sqlalchemy import select

from database import get_db
from models import DBEvent, DBCategory
from schemas import EventSchema, EventDetailSchema

router = APIRouter(prefix="/events", tags=["Events"])

# GET /events - retrieve all events
@router.get("/", response_model=List[EventSchema])
def get_events(
    skip: int = 0,
    limit: int = 100,
    db: Session = Depends(get_db)
) -> List[DBEvent]:
    """
    Retrieve a paginated list of all events.
    - skip: number of records to skip (offset)
    - limit: maximum number of records to return
    """
    stmt = select(DBEvent).offset(skip).limit(limit)
    events = db.execute(stmt).scalars().all()
    return events

# GET /events/{event_id} - retrieve a single event by ID
@router.get("/{event_id}", response_model=EventDetailSchema)
def get_event(
    event_id: int,
    db: Session = Depends(get_db)
) -> DBEvent:
    """
    Retrieve a single event by its ID.
    Returns 404 if the event does not exist.
    """
```

```

"""
stmt = select(DBEvent).where(DBEvent.id == event_id)
event = db.execute(stmt).scalar_one_or_none()

if event is None:
    raise HTTPException(
        status_code=status.HTTP_404_NOT_FOUND,
        detail=f"Event with id={event_id} not found"
    )

return event

```

Concept

The **APIRouter** organises related endpoints into a separate module — the **prefix="/events"** means all routes in this router are prefixed with **/events** automatically. **Depends(get_db)** is FastAPI's **dependency injection** mechanism — it calls **get_db()** before the endpoint handler and injects the resulting **Session** object as the **db** parameter. The **skip** and **limit** parameters with default values become optional **query parameters** (e.g., **GET /events?skip=0&limit=10**) — FastAPI handles parsing and type conversion automatically. **response_model=List[EventSchema]** tells FastAPI to serialize the returned list through the Pydantic schema, stripping any fields not declared in **EventSchema**. **scalar_one_or_none()** returns the single result or **None** — never raises an exception, making the **None** check clean. **HTTPException** with 404 is the REST-correct way to signal a missing resource.

Why This Approach?

Pagination via **skip/limit** query parameters is essential for any endpoint that might return large datasets. Without pagination, a **GET /events** on a table with a million rows would load all million rows into memory and send a massive response. The **offset/limit** SQL pattern returns only the requested page. **Depends(get_db)** is the FastAPI idiom for resource management — it cleanly handles session lifecycle without polluting the endpoint logic. The **scalar_one_or_none() + HTTPException(404)** pattern is the correct REST approach: return 404 for a missing resource, not an empty 200 or a 500.

Expected Output & Explanation

GET /events?skip=0&limit=10 response:

```
[{"id":1, "product_code":"SP100", "date":"2024-06-15", "price":"99.99", "category_id":1}, ...]
```

GET /events/1 response:

```
{"id":1, "product_code":"SP100", "date":"2024-06-15", "price":"99.99", "category_id":1,
"category":{"id":1,"name":"Sports"}}
```

GET /events/999 response: 404 {"detail": "Event with id=999 not found"}

Cell 8 — POST, PUT, and DELETE Endpoints — Full CRUD API

```
# Continued in routers/events.py
from schemas import EventCreateSchema, EventUpdateSchema

class EventCreateSchema(BaseModel):
    product_code: str
    date: date
    price: Decimal
    category_id: int

class EventUpdateSchema(BaseModel):
    date: date | None = None # optional: only update provided fields
    price: Decimal | None = None

# POST /events - create a new event
@router.post("/", response_model=EventSchema,
status_code=status.HTTP_201_CREATED)
def create_event(
    payload: EventCreateSchema,
    db: Session = Depends(get_db)
) -> DBEvent:
    # Verify the category exists
    category = db.get(DBCategory, payload.category_id)
    if category is None:
        raise HTTPException(status_code=400, detail="Category not found")

    new_event = DBEvent(**payload.model_dump())
    db.add(new_event)
    db.commit()
    db.refresh(new_event) # reload from DB to get auto-generated id
    return new_event
```

```
# PATCH /events/{event_id} - partial update
@router.patch("/{event_id}", response_model=EventSchema)
def update_event(
    event_id: int,
    payload: EventUpdateSchema,
    db: Session = Depends(get_db)
) -> DBEvent:
    event = db.get(DBEvent, event_id)
    if event is None:
        raise HTTPException(status_code=404, detail="Event not found")

    # Update only the fields that were provided
    update_data = payload.model_dump(exclude_unset=True)
    for field, value in update_data.items():
        setattr(event, field, value)

    db.commit()
    db.refresh(event)
    return event

# DELETE /events/{event_id} - remove an event
@router.delete("/{event_id}", status_code=status.HTTP_204_NO_CONTENT)
def delete_event(
    event_id: int,
    db: Session = Depends(get_db)
) -> None:
    event = db.get(DBEvent, event_id)
    if event is None:
        raise HTTPException(status_code=404, detail="Event not found")

    db.delete(event)
    db.commit()
```

Concept

The `status_code=HTTP_201_CREATED` on the POST endpoint correctly signals resource creation — this is more precise than a generic 200 OK. `payload.model_dump()` converts the Pydantic model to a plain dictionary that can be unpacked as keyword arguments to `DBEvent(**...)` — constructing the ORM object from validated data. `db.refresh(new_event)` reloads the object from the database after the commit, populating auto-generated fields like `id` that SQLite assigns during INSERT. The PATCH endpoint uses `model_dump(exclude_unset=True)` — this returns only the fields that were **explicitly provided** in the request body, excluding fields left at their defaults. This is the correct partial-update pattern — without `exclude_unset`, unset optional fields would overwrite existing data with `None`. The DELETE endpoint returns `204 No Content` — the REST convention for a successful deletion with no response body.

Why This Approach?

The `exclude_unset=True` pattern is one of the most important Pydantic/FastAPI idioms for PATCH endpoints. It means `{"price": 149.99}` only updates the price — the date remains unchanged. Without it, the client would be forced to send all fields in every update request, which is the semantics of PUT (full replacement). `db.refresh()` after commit is necessary because SQLAlchemy's identity map may have stale data — refreshing ensures the returned object reflects the actual database state including any auto-generated or trigger-modified values.

Expected Output & Explanation

POST /events body: `{"product_code": "SP300", "date": "2024-09-01", "price": "149.99", "category_id": 1}`
 Response 201: `{"id": 3, "product_code": "SP300", "date": "2024-09-01", "price": "149.99", "category_id": 1}`
 PATCH /events/3 body: `{"price": "199.99"}`
 Response 200: `{"id": 3, ... "price": "199.99", "date": "2024-09-01"}` (date unchanged)
 DELETE /events/3: Response 204 No Content (empty body, success)

Part D — Unit Testing with pytest and TestClient

Cell 9 — Writing and Running API Tests with pytest

```
# tests/test_events.py
import pytest
from fastapi.testclient import TestClient
from sqlalchemy import create_engine
from sqlalchemy.orm import sessionmaker

from main import app
```



```
from models import Base
from database import get_db

# --- Test database: in-memory SQLite ---
TEST_DATABASE_URL = "sqlite:///memory:"

test_engine = create_engine(
    TEST_DATABASE_URL,
    connect_args={"check_same_thread": False}
)
TestingSessionLocal = sessionmaker(bind=test_engine)

# --- Override the database dependency for tests ---
def override_get_db():
    db = TestingSessionLocal()
    try:
        yield db
    finally:
        db.close()

app.dependency_overrides[get_db] = override_get_db

# --- Fixtures ---
@pytest.fixture(autouse=True)
def setup_database():
    """Create tables before each test, drop them after."""
    Base.metadata.create_all(bind=test_engine)
    yield
    Base.metadata.drop_all(bind=test_engine)

# Test client - sends HTTP requests to the API without a real server
client = TestClient(app)
```

```
# --- Test: health check ---
def test_home():
    response = client.get("/")
    assert response.status_code == 200
    assert response.json()["message"] == "Events API is running"

# --- Test: create and retrieve an event ---
def test_create_and_get_event():
    # First, create a category
    cat_res = client.post("/categories/", json={"name": "Sports"})
    assert cat_res.status_code == 201
    cat_id = cat_res.json()["id"]

    # Create an event in that category
    event_payload = {
        "product_code": "SP100",
        "date": "2024-06-15",
        "price": "99.99",
        "category_id": cat_id
    }
    ev_res = client.post("/events/", json=event_payload)
    assert ev_res.status_code == 201
    event_id = ev_res.json()["id"]

    # Retrieve the event
    get_res = client.get(f"/events/{event_id}")
    assert get_res.status_code == 200
    data = get_res.json()
    assert data["product_code"] == "SP100"
    assert data["price"] == "99.99"

# --- Test: 404 for non-existent event ---
def test_get_nonexistent_event():
    response = client.get("/events/9999")
    assert response.status_code == 404
    assert "not found" in response.json()["detail"].lower()
```

Concept

The `TestClient` wraps the FastAPI application and sends HTTP requests **in-process** — no real HTTP server is needed. `app.dependency_overrides[get_db] = override_get_db` is FastAPI's dependency injection override mechanism — it replaces the production database session with a test session pointing to an in-memory SQLite database. The `@pytest.fixture(autouse=True)` decorator makes the `setup_database` fixture run automatically before **every test function** — creating a fresh database schema before each test and dropping it after, guaranteeing test isolation. The `yield` in the fixture marks the point where the test runs — code before `yield` is setup, code after is teardown. `assert response.status_code == 201` is the fundamental pattern of HTTP API testing — verify the correct status code, then verify the response body.

Why This Approach?

In-memory SQLite for tests is the gold standard for fast, isolated API testing. Each test gets a brand-new database — no cleanup needed, no test interference. The dependency override pattern is FastAPI-specific and elegant: it allows swapping any dependency (database, email service, cache) with a test double without modifying production code. Tests should be **fast, isolated, and deterministic** — this setup achieves all three. Running `poetry run pytest` discovers and executes all `test_*.py` files automatically.

Expected Output & Explanation

pytest output (all passing):

```
tests/test_events.py::test_home PASSED
tests/test_events.py::test_create_and_get_event PASSED
tests/test_events.py::test_get_nonexistent_event PASSED
3 passed in 0.42s
```

Part E — Filesystem Operations, Regex, and YAML Parsing

Cell 10 — Reading Files from the Filesystem with Pathlib

```
from pathlib import Path

# --- Path construction ---
# Absolute path of the current file
current_file = Path(__file__).resolve()
project_root = current_file.parent.parent
data_dir      = project_root / "data"    # / operator joins paths
```

```

print(f"Project root : {project_root}")
print(f"Data dir      : {data_dir}")
print(f"Dir exists    : {data_dir.exists()}")

# --- Create directory if it doesn't exist ---
data_dir.mkdir(parents=True, exist_ok=True)

# --- Discover all YAML files in the data directory ---
yaml_files = list(data_dir.glob("*.yaml"))
print(f"\nYAML files found: {len(yaml_files)}")
for f in yaml_files:
    print(f"  {f.name} | size: {f.stat().st_size} bytes")

# --- Read a text file ---
config_file = project_root / "config.yaml"
if config_file.exists():
    content = config_file.read_text(encoding="utf-8")
    print(f"\nConfig file content ({len(content)} chars):")
    print(content[:200]) # first 200 characters

# --- Recursive glob: find all yaml files in subdirectories ---
all_yaml = list(project_root.glob("**/*.yaml"))
print(f"\nTotal YAML files (all dirs): {len(all_yaml)}")

```

Concept

The `Path` object is constructed from a string or another `Path` using the `/` operator for joining — `project_root / "data"` is the Pathlib equivalent of `os.path.join(project_root, 'data')` but far more readable. `Path(__file__).resolve()` gives the absolute path of the current source file — `.parent` moves up one directory level. `.glob("*.yaml")` matches all files ending in `.yaml` in that directory (not recursively). `**/*.yaml` is a recursive glob — `**` matches any number of directory levels. `.read_text()` reads the entire file as a string. `.stat().st_size` returns the file size in bytes. `mkdir(parents=True, exist_ok=True)` creates the directory and any missing parent directories without raising an error if it already exists.

Why This Approach?

Pathlib's object-oriented approach makes filesystem code significantly more readable and cross-platform than the `os.path` string API. The `/` operator for path joining is intuitive

and prevents common errors like missing separators or double slashes.

`Path(__file__).resolve().parent.parent` is the robust, deployment-independent way to find the project root — it works regardless of from which directory the application is launched. In a Data Science context, Pathlib is essential for discovering dataset files, managing model artifact directories, and loading configuration files.

Expected Output & Explanation

Project root : /home/user/projects/events-api

Data dir : /home/user/projects/events-api/data

Dir exists : True (after mkdir)

YAML files found: 3 (example: events-001.yaml, events-002.yaml, events-003.yaml)

events-001.yaml | size: 248 bytes

Total YAML files (all dirs): 5 (including subdirectories)

Cell 11 — Parsing Filenames with Regular Expressions

```
import re
from pathlib import Path

# --- Basic regex patterns used in the project ---

# Pattern: extract event ID from filename like 'EVENT-100.yaml'
pattern = r"EVENT-(\d+)"

test_strings = [
    "EVENT-100.yaml",
    "EVENT-2048.yaml",
    "not-an-event.yaml",
    "event-100.yaml",      # lowercase - won't match
    "EVENT-ABC.yaml",     # letters, not digits - won't match
]

print("Regex matching results:")
for text in test_strings:
    match = re.search(pattern, text)
    if match:
        event_id = match.group(1)  # captured group 1: the digits
        print(f"  MATCH | '{text}' -> event_id='{event_id}'")
```

```
else:
    print(f" NO MATCH | '{text}'")

# --- Practical: parse all event YAML files in a directory ---
data_dir = Path("data")

def extract_event_id_from_filename(filename: str) -> int | None:
    """
    Extracts the numeric event ID from a filename like 'EVENT-100.yaml'.
    Returns None if the filename does not match the expected pattern.
    """
    match = re.search(r"EVENT-(\d+)", filename, re.IGNORECASE)
    if match:
        return int(match.group(1))
    return None

# Process files: only handle files matching the event pattern
for yaml_file in data_dir.glob("*.yaml"):
    event_id = extract_event_id_from_filename(yaml_file.name)
    if event_id is not None:
        print(f" Processing event file: {yaml_file.name} ->
id={event_id}")
    else:
        print(f" Skipping unrecognized file: {yaml_file.name}")
```

Concept

The regex pattern `r"EVENT-(\d+)"` uses: a **raw string** (`r"..."`) to prevent Python from interpreting `\d` as an escape sequence; `EVENT-` as a literal string that must be present; `(\d+)` as a **capturing group** — `\d` matches any digit, `+` means one or more. The parentheses capture the matching digits so `match.group(1)` retrieves just the numeric portion. `re.search()` looks for the pattern **anywhere** in the string (vs. `re.match()` which only matches at the start). The `re.IGNORECASE` flag makes the match case-insensitive — `event-100.yaml` would also match when the flag is set. Converting `match.group(1)` to `int` gives a proper integer event ID for database queries.

Why This Approach?

The pattern of combining `Pathlib.glob()` with `re.search()` is a robust, two-stage file processing pipeline: glob narrows down the file types (only `.yaml` files), regex extracts structured data from the filename. This is far more reliable than string slicing (`filename[6:-5]`) which would fail for filenames of unexpected length. The `re.IGNORECASE` flag ensures the pattern works with either `EVENT-100.yaml` or `event-100.yaml` — important when files come from different systems with different naming conventions.

Expected Output & Explanation

Regex matching results:

MATCH | 'EVENT-100.yaml' -> event_id='100'

MATCH | 'EVENT-2048.yaml' -> event_id='2048'

NO MATCH | 'not-an-event.yaml'

NO MATCH | 'event-100.yaml' (case-sensitive: uppercase EVENT required without `re.IGNORECASE`)

NO MATCH | 'EVENT-ABC.yaml' (`\d+` requires digits, not letters)

Cell 12 — Parsing YAML Files with PyYAML

```
import yaml

from pathlib import Path
from datetime import date
from decimal import Decimal

# --- Example YAML event file (data/EVENT-100.yaml) ---
# product_code: SP100
# date: 2024-06-15
# price: 99.99
# category: Sports
# description: Premier League Match - Manchester vs Arsenal
# tags:
#   - football
#   - premier-league
#   - london

def load_event_from_yaml(file_path: Path) -> dict:
    """
    Loads and parses an event YAML file into a Python dictionary.
    Uses safe_load to prevent arbitrary object instantiation.
```

```
"""
    with file_path.open("r", encoding="utf-8") as f:
        data = yaml.safe_load(f)
    return data

def bulk_import_events_from_yaml(data_dir: Path) -> list[dict]:
    """
    Discovers all EVENT-*.yaml files and loads them into a list.
    Returns only valid files; logs and skips malformed files.
    """
    events = []
    import re
    for yaml_file in sorted(data_dir.glob("EVENT-*.yaml")):
        try:
            event_data = load_event_from_yaml(yaml_file)
            # Validate required fields
            required_fields = {"product_code", "date", "price", "category"}
            if not required_fields.issubset(event_data.keys()):
                print(f" [WARN] Missing fields in {yaml_file.name}: "
                      f"{required_fields - event_data.keys()}")
                continue
            events.append(event_data)
            print(f" [OK] Loaded: {yaml_file.name} ->
{event_data['product_code']}")
        except yaml.YAMLError as e:
            print(f" [ERROR] Malformed YAML in {yaml_file.name}: {e}")
    return events

# --- Demo ---
data_dir = Path("data")
events = bulk_import_events_from_yaml(data_dir)
print(f"\nTotal events loaded: {len(events)}")
if events:
    first = events[0]
    print(f"First event: {first['product_code']} | {first['date']} |
${first['price']}")
```



```
print(f"Tags: {first.get('tags', [])}")
```

Concept

The `yaml.safe_load()` function parses the YAML content and returns native Python objects: scalars (strings, integers, floats, booleans, dates) map to their Python equivalents, YAML sequences (`- item`) map to Python lists, and YAML mappings map to Python dictionaries. Never use `yaml.load()` without the `Loader=` argument on untrusted input — it can execute arbitrary Python code if the YAML contains `!!python/object` tags. `yaml.safe_load()` restricts parsing to safe scalar types. The `required_fields.issubset()` validation uses set arithmetic to check that all required keys are present in the loaded dictionary. The `yaml.YAMLError` exception catches syntax errors in malformed YAML files — the `try/except` with `continue` ensures one bad file doesn't stop processing the rest.

Why This Approach?

The YAML import pipeline combines multiple skills: Pathlib for file discovery, PyYAML for parsing, set operations for validation, and exception handling for robustness. This is a realistic **data ingestion pattern** — in production systems, events are often authored as YAML files by non-technical stakeholders and imported into the database in batch. The set-based field validation is Pythonic and concise: `required_fields - event_data.keys()` gives the set of missing fields in one expression. Using `sorted()` on the glob result ensures files are processed in a deterministic, alphabetical order — important for reproducible imports.

Expected Output & Explanation

```
[OK] Loaded: EVENT-100.yaml -> SP100
[OK] Loaded: EVENT-200.yaml -> SP200
[WARN] Missing fields in EVENT-300.yaml: {'price'}
Total events loaded: 2 (EVENT-300 skipped due to missing field)
First event: SP100 | 2024-06-15 | $99.99
Tags: ['football', 'premier-league', 'london'] (list from YAML sequence)
```

Quick Reference — REST API Stack

Task	Library & Pattern
Create FastAPI app	<code>app = FastAPI(title=..., version=...)</code>
Define route	<code>@app.get("/path") / @app.post("/path", status_code=201)</code>
Path parameter	<code>def endpoint(item_id: int)</code> — auto-parsed from URL

Query parameter	<code>def endpoint(skip: int = 0, limit: int = 100) —</code> auto-parsed from URL
Dependency injection	<code>db: Session = Depends(get_db) —</code> session per request
Request body validation	<code>Pydantic BaseModel + model_config =</code> <code>ConfigDict(from_attributes=True)</code>
Response model	<code>@app.get("/", response_model=EventSchema)</code>
Return 404	<code>raise HTTPException(status_code=404, detail="Not</code> <code>found")</code>
Partial update	<code>payload.model_dump(exclude_unset=True) +</code> <code>setattr(obj, k, v)</code>
Test client	<code>from fastapi.testclient import TestClient; client =</code> <code>TestClient(app)</code>
Override dependency	<code>app.dependency_overrides[get_db] = override_get_db</code>
Load .env	<code>load_dotenv(); os.getenv('KEY', 'default')</code>
Discover files	<code>Path('data').glob('*.yaml')</code>
Join paths	<code>base / 'subdir' / 'file.yaml'</code>
Regex match	<code>re.search(r'EVENT-(\d+)', text); match.group(1)</code>
Parse YAML	<code>yaml.safe_load(file_handle) →</code> Python dict
Run server	<code>poetry run uvicorn main:app --reload</code>
Run tests	<code>poetry run pytest -v</code>
Type check	<code>poetry run mypy .</code>

Chapter Summary

This chapter walked through building a complete, production-grade REST API in Python, covering every layer of the stack:

1. **REST Principles** — HTTP methods (GET, POST, PUT, PATCH, DELETE), status codes, and resource-oriented URL design. REST is the foundation on which all modern web APIs are built.
2. **Poetry** — Unified dependency management with `pyproject.toml` (declaration) and `poetry.lock` (reproducibility). The standard for professional Python project setup.
3. **Environment Configuration** — The `.env` pattern with `python-dotenv` keeps secrets out of source control. Fail-fast validation ensures missing configuration is caught at startup, not buried in runtime errors.
4. **Database Design** — Relational schema with primary keys, UNIQUE constraints, NOT NULL, foreign keys, NUMERIC types for money, and indexes for performance.

5. **SQLAlchemy ORM Models** — DBCategory and DBEvent with Mapped type annotations, one-to-many relationships, and cascading deletes. The DB prefix convention separates models from schemas.
6. **FastAPI Application** — Application factory, startup events, APIRouter for modular endpoint organisation, Depends() for dependency injection, full CRUD endpoints with correct HTTP semantics.
7. **Pydantic Schemas** — Separation of DB models from API schemas, from_attributes=True for ORM bridge, exclude_unset=True for partial updates, nested schemas for detail views.
8. **Testing** — pytest with FastAPI TestClient, in-memory SQLite for test isolation, autouse fixtures for setup/teardown, dependency_overrides for swapping the real database.
9. **Pathlib + Regex + YAML** — Object-oriented filesystem access, pattern-based file discovery, regex-based filename parsing, and safe YAML parsing for bulk data import pipelines.

What's Next?

In the next chapter, we will build on this REST API foundation by adding authentication and authorization (JWT tokens and OAuth2), advanced Pydantic validation with custom validators, background task processing, and deploying the application using Docker and a CI/CD pipeline — the complete production engineering picture.